

A Denotational Semantics for λ_C with Parameterized Handlers

Abstract

We give a denotational semantics for a fragment of the effect calculus λ_C : an expression language with algebraic effect operations, a distinguished **loss** effect used to record real-valued costs, and effect handlers **with** h **from** e_1 **handle** e_2 that thread a running *parameter* across successive resumptions of a handled computation’s delimited continuation. Types are interpreted by a family of selection functions $\widehat{S}_\epsilon(X) = (X \rightarrow \widehat{R}_\epsilon) \rightarrow R \times \widehat{Step}_\epsilon(X)$, built over an ordinary, loss-free free monad F_ϵ of ground-typed effect trees. Unlike a construction that attaches a loss contribution to every node of a single tree monad, here the accumulated loss is produced exactly once at each step of unfolding a selection function, entirely separately from the pending-computation tree, and a node’s own children are themselves selection functions rather than bare trees. We develop F_ϵ , $\widehat{S}_\epsilon$, the semantics of expressions, and, in particular, the semantics of parameterized handlers, which we define by a direct structural recursion on a selection function’s own step-by-step unfolding – dispatching on whether the next pending operation belongs to the handler being applied – rather than by extending an algebra map homomorphically. We motivate this choice: once handlers carry a running parameter, a clause may resume its own delimited continuation at a parameter different from the one under which the handler was itself invoked, and a nested handler’s own choice continuation may leak a value computed under such a resumption into an enclosing computation; together these defeat the uniform-in-the- parameter homomorphism property that an extension-based account would need, whereas a direct recursion on one pending step at a time requires no such property.

1 Introduction

λ_C interprets terms via a family of *selection functions*

$$\widehat{S}_\epsilon(X) = (X \rightarrow \widehat{R}_\epsilon) \rightarrow R \times \widehat{Step}_\epsilon(X),$$

for R the (additive) monoid in which losses are recorded. Intuitively, a value of $\widehat{S}_\epsilon(X)$ takes a *loss continuation* $\gamma : X \rightarrow \widehat{R}_\epsilon$ – an estimate of the eventual loss incurred by each possible result – and, run against it, produces a pair: the loss $r \in R$ incurred at this one step of the computation, together with either a final value or one further pending operation call, whose own continuations are again selection functions run against the same γ . Two design choices distinguish this construction from one in which the accumulated loss is instead attached to every node of a single effect-tree monad, with handler semantics obtained as the (unique) homomorphic extension of an algebra map:

1. The accumulated loss is kept entirely separate from the pending-computation tree, as a plain pair produced once at each step of unfolding a selection function, rather than woven into the tree’s own recursive structure. The auxiliary monad F_ϵ used to represent finished loss *reports* is consequently an entirely ordinary, loss-free free monad: no loss is attached to any of its own nodes, at all.

2. Handler semantics is defined by a direct structural recursion on a selection function's own step-by-step unfolding, dispatching on whether the next pending operation belongs to the handler being applied, rather than by first building an algebra structure and appealing to its universal (homomorphic-extension) property.

The second choice matters once handlers carry a running parameter. A handler's clause may resume its own delimited continuation at a parameter *different* from the one under which the handler was itself invoked – choosing, at each resumption, both a fresh parameter value and a point to resume at. Independently, a nested handler's own choice continuation – the value it reports back describing what loss it *would* incur, were a given resumption selected – may itself leak a value computed under such a resumption into an enclosing computation. Together, these two features defeat the uniform-in-the-parameter algebra-homomorphism property that an extension-based account of handler semantics would need: the map obtained by extending an algebra homomorphically is only guaranteed to behave well when it is applied *uniformly*, and a resumption at a freshly chosen parameter is precisely a non-uniform use of the extension. A direct recursion on one pending step at a time never needs to state, let alone prove, any such uniform property in the first place: each case of the recursion only ever inspects the selection function's immediately next step, dispatches or recurses into a strictly smaller subtree, and the whole definition is well founded by ordinary structural recursion on that unfolding – mirroring, step for step, how a handled computation is itself reduced operationally, one pending operation at a time.

As with any construction that restricts attention to *ground* (first-order) operation signatures – no operation may take or return a value of function type, though the object language's expressions remain fully higher-order throughout, and handler clauses may themselves return functions – this keeps the auxiliary monad F_ϵ an ordinary, non-mutually-recursive inductive definition, needing only the ground-type interpretation of types, not the general one (which is defined mutually with $\hat{S}_\epsilon$ itself, via the interpretation of function types).

Section 2 recalls the syntax and semantics of types, including parameterized handlers. Section 3 develops the monads F_ϵ and $\hat{S}_\epsilon$. Section 4 gives the semantics of expressions. Section 5 gives the semantics of handlers.

2 Syntax and Semantics of Types

Types σ, τ are built from ground (first-order) base types, tuples, and function types $\sigma \rightarrow \tau! \epsilon$ decorated with an effect context ϵ (a multiset of effect labels available to the function body, written additively: $\epsilon(\ell)$ counts the occurrences of ℓ in ϵ , and $\epsilon\ell$ denotes ϵ extended by one further, outermost occurrence of ℓ). A signature Σ fixes, for each effect label ℓ , a family of operations $op : out \xrightarrow{\ell} in$ with out, in ground types, and a distinguished ground type **loss** of loss constants $r \in R$.

Values include variables, ground constants (base constants, loss constants $r : \mathbf{loss}$, and $()$), pairs, and abstractions $\lambda^\epsilon x : \sigma. e$. Expressions e include values, applications of a primitive (first-order) function, tuples, projections, function application, an operation call $op(e)$ (typed against a witness that $\ell \in \epsilon$), **loss**(e) to record a loss, a $(\blacktriangleright)$ -construct $e_1 \blacktriangleright \lambda^{\epsilon_1} x : \sigma_1. e_2$ that sequences a hypothetical loss continuation, a local construct $\langle e \rangle_g^{\epsilon_1}$ that substitutes a different loss continuation while evaluating e , **reset** e to discard recorded loss, and handling **with** h **from** e_1 **handle** e_2 .

A handler h for effect label ℓ , parameter type par , body type σ , and result type σ' at ambient effect context ϵ , supplies, for each operation $op : out \xrightarrow{\ell} in$, a *clause* e_{op} , typed in a context extended by four variables – the *running parameter* $z_p : par$, the operation's own argument $z_v : out$, a *choice continuation* $l : (par \times in) \Rightarrow \mathbf{loss}! \epsilon$, and a *delimited continuation* $k : (par \times in) \Rightarrow \sigma'! \epsilon$ – of

result type $\sigma'!\epsilon$; and a *return clause* e_r , typed in a context extended by the running parameter $z_p:par$ and a result variable $z_x:\sigma$, likewise of type $\sigma'!\epsilon$. Both l and k take a *pair* (p', a) of a freshly chosen parameter value and an operation-result value: a clause resumes the handled computation by supplying, at each resumption, both the parameter the handler should continue running under and the point in the operation's result type to resume at. In **with** h **from** e_1 **handle** e_2 , the expression $e_1 : par!\epsilon$ supplies the *initial* parameter value under which h begins running $e_2 : \sigma!\epsilon\ell$.

The semantics of types, $S[\sigma]$, is given by

$$S[b] = \llbracket b \rrbracket, \quad S[(\sigma_1, \dots, \sigma_n)] = S[\sigma_1] \times \dots \times S[\sigma_n], \quad S[\sigma \rightarrow \tau!\epsilon] = S[\sigma] \rightarrow \widehat{S}_\epsilon(S[\tau]),$$

where $\widehat{S}_\epsilon$ is the selection-function family developed in Section 3.

3 The Monads F_ϵ and $\widehat{S}_\epsilon$

3.1 The carrier F_ϵ

An ϵ -algebra is a set X with maps $\varphi_{\ell, op, i} : S[\text{out}] \times X^{S[\text{in}]} \rightarrow X$ for each $\ell \in \epsilon$, $op : out \xrightarrow{\ell} in$, $0 < i \leq \epsilon(\ell)$; the free such algebra on a set X is written $F_\epsilon(X)$.

Definition 1 (The monad F_ϵ). *For a set X , let $F_\epsilon(X)$ be the least set Y such that*

$$Y = X + \sum_{\substack{\ell \in \epsilon, \text{ op: out } \xrightarrow{\ell} \text{ in} \\ 0 < i \leq \epsilon(\ell)}} S[\text{out}] \times Y^{S[\text{in}]}.$$

We write an element as either a leaf $\iota_1(x)$ for $x \in X$, or an internal node $\iota_2((\ell, op, i), (o, \kappa))$; F_ϵ is the ordinary free monad for the ϵ -signature, with unit $\eta^{F_\epsilon}(x) = \iota_1(x)$, algebra structure $\varphi_{\ell, op, i}(o, \kappa) = \iota_2((\ell, op, i), (o, \kappa))$, and Kleisli extension $f : F_\epsilon(X) \rightarrow F_\epsilon(Y)$ for $f : X \rightarrow F_\epsilon(Y)$ given, as usual, by structural recursion:

$$f(\iota_1(x)) = f(x), \quad f(\iota_2((\ell, op, i), (o, \kappa))) = \iota_2((\ell, op, i), (o, f \circ \kappa)).$$

No loss is attached to any node of $F_\epsilon(X)$ at all: unlike a construction in which the accumulated loss is embedded throughout a single tree monad, here F_ϵ is kept entirely free of R , and is used solely to represent finished computations – values, or trees of pending operation calls with no further loss bookkeeping of their own.

For $sub : \epsilon_1 \subseteq \epsilon$ (every occurrence available in ϵ_1 is also available in ϵ), write $(sub)_* : F_{\epsilon_1}(X) \rightarrow F_\epsilon(X)$ for the evident structural relabelling of occurrence witnesses, leaving values and tree shape untouched.

We write $\widehat{R}_\epsilon := F_\epsilon(R)$ for the type of *loss reports*: a finished computation whose leaves carry an accumulated loss value rather than an ordinary result.

3.2 Selection functions $\widehat{S}_\epsilon$

Definition 2 (The mutual family $\widehat{S}_\epsilon, \widehat{Step}_\epsilon$). *For a set X , let $\widehat{S}_\epsilon(X)$ and $\widehat{Step}_\epsilon(X)$ be given by the simultaneous least solution of*

$$\widehat{S}_\epsilon(X) = (X \rightarrow \widehat{R}_\epsilon) \rightarrow R \times \widehat{Step}_\epsilon(X), \quad \widehat{Step}_\epsilon(X) = X + \sum_{\substack{\ell \in \epsilon, \text{ op: out } \xrightarrow{\ell} \text{ in} \\ 0 < i \leq \epsilon(\ell)}} S[\text{out}] \times \widehat{S}_\epsilon(X)^{S[\text{in}]}.$$

As with F_ϵ we write an element of $\widehat{\text{Step}}_\epsilon(X)$ as either $\text{leaf}(x)$ or $\text{node}((\ell, \text{op}, i), (o, \kappa))$; since $\widehat{S}_\epsilon(X)$ is a function type, we apply it directly, writing $p(\gamma)$ for a selection function p run against a loss continuation γ .

Remark 3. $\widehat{\text{Step}}_\epsilon(X)$ is defined by exactly the same equation as one unfolding of F_ϵ 's own defining equation – except that its recursive occurrences are of $\widehat{S}_\epsilon(X)$ itself, not of $\widehat{\text{Step}}_\epsilon(X)$. A node's children are therefore further selection functions, not bare trees: unfolding one step of $p \in \widehat{S}_\epsilon(X)$ against γ yields either a finished value or one pending operation call, whose continuations must again be run against a (possibly different) loss continuation to make further progress. This is the structural feature that makes $\widehat{S}_\epsilon$ genuinely different from a family built as $(X \rightarrow \widehat{R}_\epsilon) \rightarrow F_\epsilon(R \times X)$ or similar: there, once a loss continuation is supplied, the resulting tree is an ordinary, loss-free value of a plain monad, independent of $\widehat{S}_\epsilon$ itself. Here the recursive structure of $\widehat{S}_\epsilon$ and $\widehat{\text{Step}}_\epsilon$ never bottoms out into a monad independent of $\widehat{S}_\epsilon$; the accumulated loss R that does appear is produced once per step of the unfolding, sitting entirely outside $\widehat{\text{Step}}_\epsilon(X)$, rather than being threaded through the recursive datatype at all.

One consequence: because a selection function's children are again selection functions rather than plain trees, $\widehat{S}_{\epsilon_1}(X)$ cannot simply be reinterpreted as an instance of $\widehat{S}_\epsilon(X)$ the way $F_{\epsilon_1}(X)$ can be relabelled into $F_\epsilon(X)$ by $(\text{sub})_*$: changing the ambient effect context changes the domain of every loss continuation nested inside it. Section 3.3 gives the genuine combinator, **glocal**, that this requires.

3.3 The combinators of $\widehat{S}_\epsilon$

Unit and operation calls.

$$\eta^{\widehat{S}_\epsilon}(x)(\gamma) = (0, \text{leaf}(x)), \quad \varphi_{\ell, \text{op}, i}(o, \kappa)(\gamma) = (0, \text{node}((\ell, \text{op}, i), (o, \kappa))).$$

Collapsing a selection function to a report. For $p \in \widehat{S}_\epsilon(X)$ and a loss continuation $\gamma : X \rightarrow \widehat{R}_\epsilon$, define $\lfloor p \rfloor_\gamma \in \widehat{R}_\epsilon$, the result of running p against γ *throughout* – at p itself and at every one of its own pending continuations – and collapsing the result into a single finished report:

$$\lfloor p \rfloor_\gamma = \begin{cases} F_\epsilon(r + -)(\gamma(x)) & p(\gamma) = (r, \text{leaf}(x)) \\ F_\epsilon(r + -)(\varphi_{\ell, \text{op}, i}(o, \lambda a. \lfloor \kappa(a) \rfloor_\gamma)) & p(\gamma) = (r, \text{node}((\ell, \text{op}, i), (o, \kappa))), \end{cases}$$

where $F_\epsilon(r + -)$ denotes the functorial action of F_ϵ applying $r + -$ at every leaf. That is: at a finished value, add this step's own loss r into whatever γ reports there; at a pending operation, leave the node itself untouched, add r into every leaf beneath it, and recurse into each child under the same γ .

Kleisli extension. For $f : X \rightarrow \widehat{S}_\epsilon(Y)$, define $f : \widehat{S}_\epsilon(X) \rightarrow \widehat{S}_\epsilon(Y)$, writing $\text{let}_{\widehat{S}_\epsilon} x \in X \text{ be } p \text{ in } f(x) := f(p)$, by first running

$$p(\lambda x. \lfloor f(x) \rfloor_\gamma) = (r_1, w)$$

and then, by cases on w ,

$$(\text{let}_{\widehat{S}_\epsilon} x \in X \text{ be } p \text{ in } f(x))(\gamma) = (r_1 + r_2, s_2), \quad (r_2, s_2) = f(x')(\gamma), \quad \text{if } w = \text{leaf}(x'),$$

$$(\text{let}_{\widehat{S}_\epsilon} x \in X \text{ be } p \text{ in } f(x))(\gamma) = (r_1, \text{node}((\ell, \text{op}, i), (o, \lambda a. \text{let}_{\widehat{S}_\epsilon} x \in X \text{ be } \kappa(a) \text{ in } f(x)))) , \\ \text{if } w = \text{node}((\ell, \text{op}, i), (o, \kappa)).$$

That is: run p under the hypothetical continuation $\lambda x. [f(x)]_\gamma$ (what f would eventually report, at each possible x , run to completion against γ); if p is already finished at some x' , the whole computation continues as $f(x')$ run directly against γ , with p 's own step-loss r_1 added on; if p has a pending operation, that operation is left exactly in place – with loss r_1 untouched – and only its children are rebound.

Adding loss. For $r \in R$, $\text{tell}(r) : \widehat{S}_\epsilon(X) \rightarrow \widehat{S}_\epsilon(X)$ adds r into a selection function's own next step, wherever that step occurs:

$$\text{tell}(r)(p)(\gamma) = (r + r_1, s_1), \quad (r_1, s_1) = p(\gamma).$$

Promoting a report. $\text{up} : F_\epsilon(X) \rightarrow \widehat{S}_\epsilon(X)$ promotes a finished, γ -independent report into a selection function that ignores whatever continuation it is later run against:

$$\text{up}(\iota_1(x))(\gamma) = (0, \text{leaf}(x)), \quad \text{up}(\iota_2((\ell, op, i), (o, \kappa)))(\gamma) = (0, \text{node}((\ell, op, i), (o, \lambda a. \text{up}(\kappa(a)))).$$

Fixing a loss continuation. For $sub : \epsilon_1 \subseteq \epsilon$ and $\gamma_1 : X \rightarrow \widehat{R}_{\epsilon_1}$, define $\text{glocal}(sub, \gamma_1, -) : \widehat{S}_{\epsilon_1}(X) \rightarrow \widehat{S}_\epsilon(X)$, which runs p against the fixed continuation γ_1 throughout – ignoring whatever γ it is itself later run against – while relabelling every occurrence witness along the way via sub :

$$\text{glocal}(sub, \gamma_1, p)(\gamma) = (r, \text{leaf}(x)), \quad \text{if } p(\gamma_1) = (r, \text{leaf}(x)),$$

$$\begin{aligned} \text{glocal}(sub, \gamma_1, p)(\gamma) &= (r, \text{node}(sub(\ell, op, i), (o, \lambda a. \text{glocal}(sub, \gamma_1, \kappa(a))))) , \\ &\quad \text{if } p(\gamma_1) = (r, \text{node}((\ell, op, i), (o, \kappa))). \end{aligned}$$

Discarding recorded loss. $\text{censor}_0 : \widehat{S}_\epsilon(X) \rightarrow \widehat{S}_\epsilon(X)$ recursively zeroes the loss recorded at every step of p 's own unfolding (run against a caller-supplied γ), leaving values and tree shape untouched:

$$\text{censor}_0(p)(\gamma) = (0, \text{leaf}(x)), \quad \text{if } p(\gamma) = (r, \text{leaf}(x)),$$

$$\begin{aligned} \text{censor}_0(p)(\gamma) &= (0, \text{node}((\ell, op, i), (o, \lambda a. \text{censor}_0(\kappa(a))))) , \\ &\quad \text{if } p(\gamma) = (r, \text{node}((\ell, op, i), (o, \kappa))). \end{aligned}$$

4 Semantics of Expressions

4.1 Values

$$\begin{aligned} \mathbb{V}[\![x]\!](\rho) &= \rho(x), & \mathbb{V}[\![c]\!](\rho) &= \llbracket c \rrbracket, & \mathbb{V}[\![(v, w)]\!](\rho) &= (\mathbb{V}[\![v]\!](\rho), \mathbb{V}[\![w]\!](\rho)), \\ \mathbb{V}[\![\lambda^\epsilon x:\sigma. e]\!](\rho) &= \lambda a. \mathbb{S}[\![e]\!](\rho[a/x]). \end{aligned}$$

4.2 The auxiliary loss-function semantics $\mathcal{L}$

For $\Gamma, x:\sigma \vdash e : \text{loss}! \epsilon$, define $\mathbb{L}[\lambda^\epsilon x:\sigma. e] : \widehat{S}_\Gamma \rightarrow \widehat{S}_\sigma \rightarrow \widehat{R}_\epsilon$ by

$$\mathbb{L}[\lambda^\epsilon x:\sigma. e](\rho)(a) = \lfloor \mathbb{V}[\lambda^\epsilon x:\sigma. e](\rho)(a) \rfloor_{\eta^{F_\epsilon}}.$$

Remark 4. The loss function $g = \lambda^\epsilon x:\sigma. e$ is itself an ordinary value of function type $\sigma \rightarrow \text{loss}! \epsilon$; its own body may call operations, sequence further loss continuations, and eventually produce a value of type loss , i.e. an $r \in R$. That eventual value is itself a further loss contribution – g is being run purely to find out what loss it would report, so whatever it eventually returns must be folded into the total, not discarded. Collapsing $\mathbb{V}[g](\rho)(a)$ under the loss continuation $\eta^{F_\epsilon} : R \rightarrow \widehat{R}_\epsilon$ (which embeds a returned loss value as itself) does exactly this, via $\lfloor - \rfloor$ ’s own leaf case ($F_\epsilon(r + -)(\gamma(x))$, here with $\gamma(x) = \eta^{F_\epsilon}(x)$). Collapsing instead under a continuation that discards its argument (e.g. $\lambda r. \eta^{F_\epsilon}(0)$) would silently drop g ’s own returned value whenever g ’s body ends in something other than a bare $\text{loss}(-)$ call – for instance a loss function that simply evaluates to a constant $r_0 : \text{loss}$ without ever calling $\text{loss}(-)$ explicitly would then be indistinguishable from one reporting no loss at all, which is not the intended reading of $\mathcal{L}$.

4.3 Core constructs

$$\mathbb{S}[v](\rho) = \eta^{\widehat{S}_\epsilon}(\mathbb{V}[v](\rho)), \quad \mathbb{S}[f(e)](\rho) = \text{let}_{\widehat{S}_\epsilon} a \in \mathbb{S}[\gamma] \text{ be } \mathbb{S}[e](\rho) \text{ in } \eta^{\widehat{S}_\epsilon}(\llbracket f \rrbracket(a)),$$

$$\mathbb{S}[(e_1, e_2)](\rho) = \text{let}_{\widehat{S}_\epsilon} a \in \mathbb{S}[\sigma_1] \text{ be } \mathbb{S}[e_1](\rho) \text{ in } \text{let}_{\widehat{S}_\epsilon} b \in \mathbb{S}[\sigma_2] \text{ be } \mathbb{S}[e_2](\rho) \text{ in } \eta^{\widehat{S}_\epsilon}((a, b)),$$

with **fst**, **snd**, and application $e_1 e_2$ the evident generic clauses of a monadic semantics ($\mathbb{S}[v e](\rho) = \text{let}_{\widehat{S}_\epsilon} a \in \mathbb{S}[\sigma] \text{ be } \mathbb{S}[e](\rho) \text{ in } \mathbb{V}[v](\rho)(a)$), and

$$\mathbb{S}[\text{op}(e)](\rho) = \text{let}_{\widehat{S}_\epsilon} a \in \mathbb{S}[\text{out}] \text{ be } \mathbb{S}[e](\rho) \text{ in } \varphi_{\ell, \text{op}, \epsilon(\ell)}(a, (\eta^{\widehat{S}_\epsilon})^{\mathbb{S}[\text{in}]}).$$

loss(e). A bare tell is directly available on $\widehat{S}_\epsilon$, so no pattern matching on the shape of $\mathbb{S}[e]$ is needed:

$$\mathbb{S}[\text{loss}(e)](\rho) = \text{let}_{\widehat{S}_\epsilon} r \in R \text{ be } \mathbb{S}[e](\rho) \text{ in } \text{tell}(r)(\eta^{\widehat{S}_\epsilon}(())).$$

$e_1 \blacktriangleright \lambda^{\epsilon_1} x:\sigma_1. e_2$. Collapse e_1 to a report using $g = \lambda x. e_2$ ’s (widened) semantics as the loss continuation throughout, then promote that report back into a selection function via **up**:

$$\mathbb{S}[e_1 \blacktriangleright \lambda^{\epsilon_1} x:\sigma_1. e_2](\rho) = \text{up}\left(\lfloor \mathbb{S}[e_1](\rho) \rfloor_{\lambda a. (sub)_*}(\mathbb{L}[\lambda x. e_2](\rho)(a))\right),$$

for $sub : \epsilon_1 \subseteq \epsilon$ (left implicit in the surface notation, as throughout). Because **up** ignores whatever continuation it is itself later run against, $\mathbb{S}[e_1 \blacktriangleright g](\rho)$ does not depend on the ambient loss continuation the surrounding computation eventually supplies – $\blacktriangleright$ has already fixed its own loss accounting via g by the time it produces a value.

$\langle e \rangle_g^{\epsilon_1}$. For $e : \sigma! \epsilon_1$ and $g : \sigma \rightarrow \text{loss}! \epsilon_2$ with $\epsilon_2 \subseteq \epsilon_1 \subseteq \epsilon$ (again left implicit), this substitutes a different loss continuation while evaluating e , using **glocal** (Section 3.3) rather than a bare application, since $\widehat{S}_{\epsilon_1}(\sigma)$ is not literally reusable as a value of $\widehat{S}_\epsilon(\sigma)$:

$$\mathbb{S}[\langle e \rangle_g^{\epsilon_1}](\rho) = \text{glocal}\left(sub_2, \lambda a. (sub_1)_*(\mathbb{L}[g](\rho)(a)), \mathbb{S}[e](\rho)\right).$$

reset e . Discard all loss recorded while evaluating e , keeping its value, via `sensor0`:

$$\mathbb{S}[\text{reset } e](\rho) = \text{sensor}_0(\mathbb{S}[e](\rho)).$$

Handling. Evaluate e_1 to obtain the initial parameter value, then dispatch to the handler semantics developed in Section 5:

$$\mathbb{S}[\text{with } h \text{ from } e_1 \text{ handle } e_2](\rho) = \text{let}_{\widehat{S}_\epsilon} p \in \mathbb{S}[par] \text{ be } \mathbb{S}[e_1](\rho) \text{ in } \mathcal{H}_h(\rho, p, \mathbb{S}[e_2](\rho)).$$

5 Semantics of Handlers

Fix a handler h for effect label ℓ , and an environment $\rho \in \widehat{S}_\Gamma$. We define $\mathcal{H}_h(\rho, p, G) \in \widehat{S}_\epsilon(\mathbb{S}[\sigma'])$ for a running parameter value $p \in \mathbb{S}[par]$ and a computation $G \in \widehat{S}_{\epsilon\ell}(\mathbb{S}[\sigma])$ still to be handled, by direct structural recursion on G 's own next unfolding step – *not* by building an $\epsilon\ell$ -algebra structure and taking a homomorphic extension.

The return clause. Write

$$s(\rho, p, a) = \mathbb{S}[e_r](\rho[p/z_p, a/z_x]) \in \widehat{S}_\epsilon(\mathbb{S}[\sigma'])$$

for the semantics of h 's return clause, at running parameter p and result value a .

The handler semantics. Running $\mathcal{H}_h(\rho, p, G)$ against a loss continuation $\gamma : \mathbb{S}[\sigma'] \rightarrow \widehat{R}_\epsilon$ first runs G itself against the hypothetical continuation $\lambda a. \iota(\lfloor s(\rho, p, a) \rfloor_\gamma)$ – what the return clause would eventually report, at each possible result a , were G to finish there – where $\iota : \widehat{R}_\epsilon \hookrightarrow \widehat{R}_{\epsilon\ell}$ is the evident inclusion induced by $\epsilon \subseteq \epsilon\ell$ (needed since G itself lives at the wider context $\epsilon\ell$). The resulting step is then dispatched by $\Delta_h(\rho, \gamma, p)$:

$$\mathcal{H}_h(\rho, p, G)(\gamma) = \Delta_h(\rho, \gamma, p)\left(G(\lambda a. \iota(\lfloor s(\rho, p, a) \rfloor_\gamma))\right).$$

Dispatch. $\Delta_h(\rho, \gamma, p)$ takes a step $(r, w) \in R \times \widehat{Step}_{\epsilon\ell}(\mathbb{S}[\sigma])$ of G 's own unfolding to a step of $\widehat{S}_\epsilon(\mathbb{S}[\sigma'])$, by cases on w :

- *G is already finished, $w = \text{leaf}(x)$:* invoke the return clause at the running parameter and the produced value, and combine the loss r_2 this incurs with whatever loss r G had already accumulated on the way to x :

$$\Delta_h(\rho, \gamma, p)(r, \text{leaf}(x)) = (r + r_2, s_2), \quad (r_2, s_2) = s(\rho, p, x)(\gamma).$$

- *G 's next step is an operation not handled by h (i.e. at an occurrence already present in ϵ , shallower than the newly introduced occurrence of ℓ that G was typed against):* pass the step through unchanged, demoting its occurrence witness, and recurse into each of its continuations:

$$\begin{aligned} \Delta_h(\rho, \gamma, p)(r, \text{node}((\ell_1, op, i), (o, \kappa))) \\ = (r, \text{node}((\ell_1, op, i), (o, \lambda a. \mathcal{H}_h(\rho, p, \kappa(a))))) , \quad \ell_1 \neq \ell. \end{aligned}$$

- *G's next step is one of h's own operations op_j , at the newly introduced occurrence: dispatch directly to the corresponding object-language clause, instantiated at the running parameter, the operation's own argument, and the choice and delimited continuations l_1, k_1 built from κ below, and run it against γ ; again combine its own freshly incurred loss r_2 with G 's own accumulated r :*

$$\Delta_h(\rho, \gamma, p)(r, \text{node}(op_j, (o, \kappa))) = (r + r_2, s_2),$$

$$(r_2, s_2) = \mathbb{S}[\![e_j]\!](\rho[p/z_p, o/z_v, l_1/l, k_1/k])(\gamma).$$

Choice and delimited continuations. At each freshly chosen parameter p' and resumption point a offered by κ , the delimited continuation k_1 resumes the same handler, running under the caller's own ambient loss continuation γ throughout (via **glocal** at the identity inclusion); the choice continuation l_1 reports what loss that same resumption *would* incur, collapsed to a bare report and promoted back into a selection function that ignores whatever continuation it is itself later run against (via $\text{up} \circ \lfloor - \rfloor_\gamma$), so that it can be used as an ordinary value of type **loss** inside the clause body:

$$k_1(p', a) = \text{glocal}(\text{id}, \gamma, \mathcal{H}_h(\rho, p', \kappa(a))), \quad l_1(p', a) = \text{up}\left(\lfloor \mathcal{H}_h(\rho, p', \kappa(a)) \rfloor_\gamma\right).$$

Remark 5. *Every recursive call above – inside Δ_h 's pass-through case, and inside k_1, l_1 – is made on $\kappa(a)$, one of G 's own children, for some a chosen by G itself: the recursion is on G 's own unfolding, one pending step at a time, and never on the running parameter p . This is exactly what lets k_1 and l_1 resume the handler at a freshly chosen p' , different at each call, with no obligation to relate the behaviour of $\mathcal{H}_h(\rho, p, -)$ and $\mathcal{H}_h(\rho, p', -)$ as p ranges over $\mathbb{S}[\![\text{par}]\!]$: nothing in the definition is stated, or needs to be proved, uniformly in p . An account built instead by first fixing a single algebra structure on $\widehat{R}_\epsilon(\mathbb{S}[\![\sigma']]\!])^{\mathbb{S}[\![\text{par}]\!]}$ and then extending a generator map homomorphically would need exactly such a uniform property – the extension's own defining property is a statement about the map's behaviour at every parameter value simultaneously – which a clause resuming at a freshly chosen $p' \neq p$, combined with a nested handler's own choice continuation reporting a value computed under such a resumption, need not respect.*